



# Pagination and infinite scrolling in React



Pagination and infinite scrolling are common techniques used to manage large datasets efficiently in web applications. Both methods aim to improve performance and user experience by only loading a subset of data at a time instead of rendering everything at once.

## Pagination in React

### What is Pagination?

Pagination involves dividing content into multiple pages, where only one page's worth of data is loaded at a time. Users can navigate between pages to view different sets of data.

### Why Use Pagination?

- **Improved Performance:**

Loads a small portion of data at once, reducing the amount of content rendered in the browser.

- **User Control:**

Users can manually navigate through different pages of content.

- **SEO Benefits:**

Each page can have its own URL and be indexed by search engines, which is helpful for larger datasets.



## How Pagination Works

### 1. Server-Side Pagination:

The server sends data based on a request with parameters like `page` and `pageSize`. The client requests only the data for a specific page.

### 2. Client-Side Pagination:

All data is fetched at once, and the client handles the pagination logic by rendering only a subset of the data.

## Example of Pagination in React (Client-Side)

```
import { useState, useEffect } from 'react';

const PaginatedList = () => {
  const [data, setData] = useState([]);
  const [currentPage, setCurrentPage] = useState(1);
  const [itemsPerPage] = useState(10);
```

Code Continue on Next Page ->



```
useEffect(() => {  
  // Fetch data from an API (this could be dynamic in a real app)  
  const fetchData = async () => {  
    const res = await  
      fetch(`https://jsonplaceholder.typicode.com/posts`);  
    const jsonData = await res.json();  
    setData(jsonData);  
  };  
  fetchData();  
}, []);  
  
// Calculate the index range for the current page  
const indexOfLastItem = currentPage * itemsPerPage;  
const indexOfFirstItem = indexOfLastItem - itemsPerPage;  
const currentItems = data.slice(indexOfFirstItem, indexOfLastItem);  
  
const paginate = (pageNumber) => setCurrentPage(pageNumber);
```

Code Continue on Next Page ->



```
return (  
  <div>  
    <ul>  
      {currentItems.map(item => (  
        <li key={item.id}>{item.title}</li>  
      ))}  
    </ul>  
    <div>  
      {Array.from({ length: Math.ceil(data.length / itemsPerPage) }, (_,  
index) => (  
        <button key={index} onClick={() => paginate(index + 1)}>  
          {index + 1}  
        </button>  
      ))}  
    </div>  
  </div>  
);  
};  
  
export default PaginatedList;
```



# Infinite Scrolling in React

## What is Infinite Scrolling?

Infinite scrolling is a technique where additional data is loaded automatically as the user scrolls down the page. This provides a smooth, continuous user experience, where the user doesn't need to manually navigate through pages.

## Why Use Infinite Scrolling?

- **Seamless User Experience:**

No need for users to click "next" or "previous" buttons. The app keeps loading content as they scroll.

- **Ideal for Dynamic Content:**

Works well for feeds, news articles, social media posts, or other types of continuous content.

- **Reduced UI Complexity:**

Removes the need for pagination controls, simplifying the UI.



## How Infinite Scrolling Works

### 1. Scroll Event Listener:

As the user scrolls down, the app detects when they've reached the bottom of the page.

### 2. API Call for More Data:

Once the bottom is reached, an API request is triggered to fetch more data.

### 3. Appending New Data:

The newly fetched data is appended to the existing content, allowing the user to scroll down further.

## Example of Infinite Scrolling in React

```
import { useState, useEffect, useCallback } from 'react';
```

```
const InfiniteScrollList = () => {  
  const [data, setData] = useState([]);  
  const [loading, setLoading] = useState(false);  
  const [page, setPage] = useState(1);
```

Code Continue on Next Page ->



```
const fetchData = useCallback(async () => {
  setLoading(true);

  const res = await
fetch(`https://jsonplaceholder.typicode.com/posts?_page=${page}&_limit=10`);

  const jsonData = await res.json();

  setData(prevData => [...prevData, ...jsonData]);

  setLoading(false);
}, [page]);

useEffect(() => {
  fetchData();
}, [fetchData]);

const handleScroll = (e) => {
  const bottom = e.target.scrollHeight === e.target.scrollTop +
    e.target.clientHeight;

  if (bottom && !loading) {
    setPage((prevPage) => prevPage + 1); // Increment page to fetch more
                                         data
  }
};
```

Code Continue on Next Page ->



```
return (  
  <div onScroll={handleScroll} style={{ height: '400px', overflowY:  
    'auto' }}>  
  
    <ul>  
      {data.map(item => (  
        <li key={item.id}>{item.title}</li>  
      ))}  
    </ul>  
  
    {loading && <p>Loading...</p>}  
  </div>  
);  
};  
  
export default InfiniteScrollList;
```



## Key Concepts in Infinite Scrolling:

- **page:**  
Tracks the current page or offset to request new data.
- **handleScroll:**  
Detects when the user scrolls to the bottom of the container.
- **fetchData:**  
Asynchronously fetches more data and appends it to the current list.
- **loading:**  
Prevents duplicate requests by ensuring new data isn't fetched while loading is in progress.



## Comparison: Pagination vs Infinite Scrolling

| Feature          | Pagination                                               | Infinite Scrolling                                                  |
|------------------|----------------------------------------------------------|---------------------------------------------------------------------|
| User Interaction | User clicks "Next" or "Previous" to change pages.        | Data loads automatically as the user scrolls.                       |
| Control          | Users have more control over where they are in the data. | Continuous flow, users don't have to manually navigate.             |
| Performance      | Can be more efficient for large datasets (server-side).  | Needs careful management of state and requests to prevent overload. |
| SEO              | Easier for search engines to crawl each page.            | Harder to manage for SEO, as content is loaded dynamically.         |
| Usability        | Good for structured datasets like product listings.      | Best for dynamic, content-heavy applications e.g., social feeds.    |